Artificial Intelligence – Assignment 2a
Zen Garden

1. Task and result

A genetic algorithm (GA) finds an order of entry points and turn decisions that lets the monk rake every free cell of the garden. On the garden from the assignment (12 × 10, 6 stones, 114 free cells), four runs saved in the project folder (all four method combinations), every population size reached 114/114.

Rules of the monk, as the assignment states them:

- He starts on the edge of the garden and rakes a straight strip until he leaves the garden or meets an obstacle (a stone or an already raked cell).
- At an obstacle he turns 90°. If both left and right are free, he chooses; if one is free, he must take it; if none is free, the game ends.
- Outside the garden he can walk freely to any other entry point.
- **Fitness** = number of raked cells. **Output** = the garden matrix with every cell numbered by the pass that raked it.
- The chromosome may hold at most half the perimeter + number of stones genes (12 + 10 + 6 = 28 for the example garden).

The program is written in Python 3.12 and consists of three files:

File	Content
`main.py`	entry point; runs the game and prints the best garden
`game.py`	input, garden, monk simulation, fitness, selection, crossover, mutation, GA loop
`fitnessVisual.py`	graphs of max, mean and min fitness per generation (matplotlib)

2. Input and garden representation

The user enters, in this order: garden width `W`, height `H`, selection method (0 = tournament, 1 = rank), mutation method (0 = random, 1 = multiplicative), number of stones, and then each stone as `x y` (x = column, y = row, both counted from 0). Any map can be entered. The configuration from the assignment, used for all experiments in this document, is:

Parameter	Value
Width W	12
Height H	10
Number of stones	6
Stones (x y)	(5, 1), (1, 2), (4, 3), (2, 4), (8, 6), (9, 6)
Free cells = optimum fitness	12 · 10 − 6 = 114
Perimeter P = 2(W + H)	44 entry points
Gene limit W + H + stones	28

Input typed into the program (tournament selection, random mutation):

```
Enter garden width: 12
Enter garden height: 10
Enter selection number 0 - tournament, 1 - rank: 0
Enter mutation number 0 - random, 1 - multiplicative: 0
Enter number of stones: 6
Enter coordinates of stone 1 (x y): 1 2
Enter coordinates of stone 2 (x y): 2 4
Enter coordinates of stone 3 (x y): 4 3
Enter coordinates of stone 4 (x y): 5 1
Enter coordinates of stone 5 (x y): 8 6
Enter coordinates of stone 6 (x y): 9 6
```

The garden is an `H × W` matrix of integers, addressed as `garden[y][x]`:

Value	Meaning
0	sand, not raked yet
−1	stone
k > 0	raked in the k-th pass of the monk

Storing the pass number instead of a plain flag serves two purposes: any value other than 0 is an obstacle for the monk, and the final matrix shows each strip and the order of the passes. The numbers 1, 2, 3, … are produced by the simulation; the input contains only sand and stones.

Stone coordinates are validated: the line must contain exactly two integers inside the garden, otherwise the program asks again. The selection and mutation numbers are asked again until they are 0 or 1.

3. Genes and chromosome

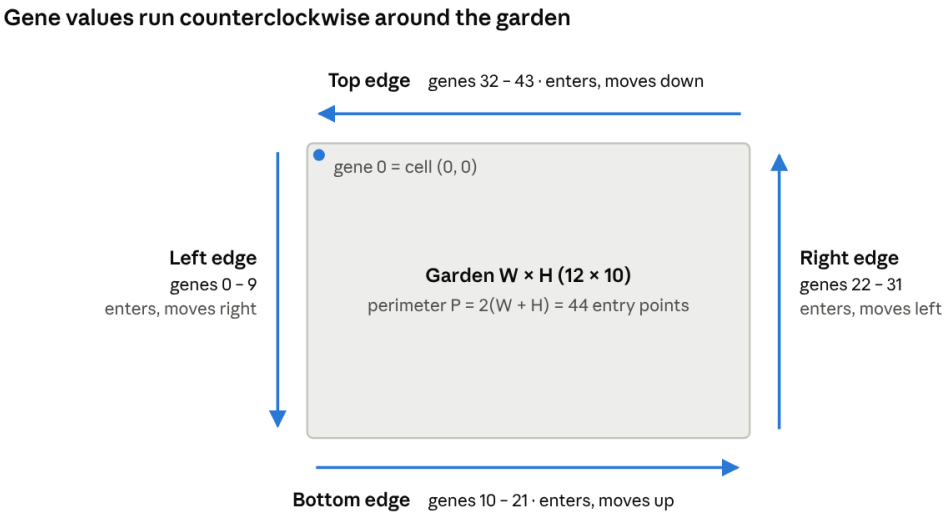
A chromosome is a list of integers. Each gene has **two roles**: it is an entry point on the perimeter, and its parity (`gene % 2`) is a possible turn decision. One list therefore encodes both the order of the passes and the monk's choices, without extra genes.

3.1 Length

Every chromosome has `max(3, int(0.85 · (W + H + stones)))` genes, i.e. **23** for the assignment garden. This is below the limit of 28, and the turn decisions add no genes. The lower bound of 3 keeps two-point crossover valid on tiny gardens. The factor 0.85 was raised from 0.75 (21 genes), since more genes tend to make performance of the chromosome better, so the solution might be found with less produced generations.

3.2 Gene value = entry point

A value `g` from `0 … P − 1` names one cell on the perimeter and the direction of entry. The numbering walks around the garden counterclockwise, so neighbouring values are neighbouring entry points:



Gene range	Edge	Start cell (x, y)	Direction
$0 \le g < H$	left	$(0, g)$	right
$H \le g < W + H$	bottom	$(g - H, H - 1)$	up
$W + H \le g < W + 2H$	right	$(W - 1, H - 1 - (g - W - H))$	left
$W + 2H \le g < P$	top	$(P - g - 1, 0)$	down

For example, gene 5 enters the left edge at row 5 moving right; gene 15 enters the bottom edge at column 5 moving up. A corner cell has two genes (gene 0 enters (0, 0) moving right, gene 43 enters it moving down); these are two different strips.

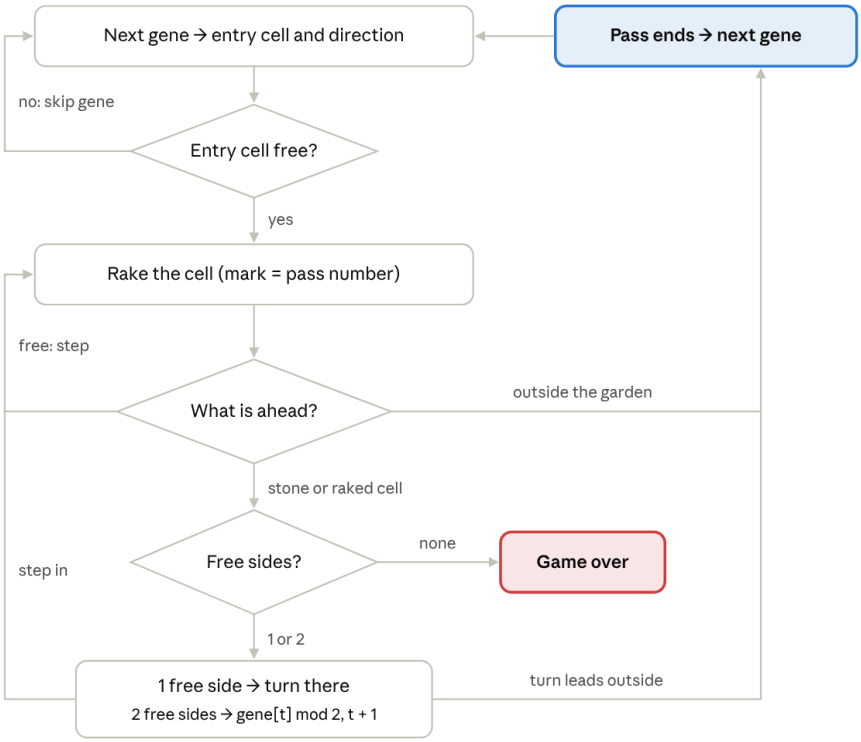
3.3 Properties of the genes

- Initial genes are uniform random integers from 0 to P − 1.
- Repeated values are allowed. A gene whose entry cell is already raked or is a stone does nothing and is skipped, so the chromosome is not a permutation and needs no repair after crossover or mutation.
- Order matters: earlier genes rake first and change which cells are free for later genes.
- **Turn decisions.** A counter `t` (0 at the start of every chromosome) points to the gene used for the next decision. When the monk must choose between two free directions, he takes option `chromosome[t mod length] mod 2` and `t` increases by one. The counter moves only at real choices, not at straight steps, forced turns or exits.
- Decoding is deterministic: the same chromosome always rakes the garden the same way and gets the same fitness. Selection can therefore compare individuals reliably, and the decisions evolve together with the entry points.

4. How the monk moves and decides

The monk is simulated by `performRake` (loop over genes) and `rakeLine` (one pass). The direction is a vector `(dx, dy)`: right (1, 0), left (−1, 0), down (0, 1), up (0, −1); y grows downwards.

One pass: the monk rakes until he leaves the garden or gets stuck



Simulation of one chromosome:

1. Copy the garden, set `passNumber = 0` and `t = 0`.
2. For each gene in order: decode the entry cell and direction (section 3). If the entry cell is not 0 (stone or raked), skip the gene. Otherwise `passNumber += 1` and run one pass.
3. **One pass** (`rakeLine`), repeated until it returns:
   - 3.1. Write `passNumber` into the current cell.
   - 3.2. Look at the next cell (x + dx, y + dy). Outside the garden → the pass ends normally. Free sand → step there and repeat.
   - 3.3. Obstacle ahead → test the two perpendicular directions (−dy, dx) and (dy, −dx). A side counts as free if its cell is sand or lies outside the garden (the monk may step out onto the edge).
   - 3.4. No free side → the monk is stuck and the whole simulation ends (game over).
   - 3.5. One free side → turn there. Two free sides → take option `chromosome[t mod length] mod 2` (0 = clockwise turn, 1 = counterclockwise), then `t += 1`.
   - 3.6. If the turned cell is outside the garden, the pass ends. Otherwise step into it and continue **in the new direction**.
4. After the last gene, or when the monk is stuck, fitness = number of cells with a value > 0.

Termination. Every iteration of a pass either leaves the garden, ends the game, or moves onto a cell that was 0 and immediately marks it. A pass therefore takes at most W · H steps, and the simulation cannot loop forever.

Example from the assignment, pass 5: the monk enters at the bottom of column 8 and goes up. At row 7 the stone at (8, 6) blocks him; both sides are free, so a gene decides → right. He continues right along row 7 to the edge and leaves, which produces the strip 5 5 5 5 in the assignment's matrix.

5. Fitness function

Fitness is the number of raked cells after the simulation of the chromosome (`computeFitness`): every cell whose value is neither 0 (sand) nor −1 (stone). The optimum is W · H − stones, i.e. 114 for the assignment garden.

- A stuck monk is not penalised separately. Getting stuck ends the simulation early, so the remaining genes rake nothing and the fitness stays lower.
- The fitness is an integer with a narrow range near the top (e.g. 105 to 114 in late generations). Raw-value selection such as roulette would barely prefer the best individuals, so the program uses rank-based methods.
- `performRake` returns both the fitness values and the raked matrices of the whole population, so the matrix of the best individual can be printed without another simulation.

6. The algorithm step by step

The program runs a generational GA with elitism, once for each population size in `POPULATIONS = [30, 50, 100]`, and keeps the best garden over all runs.

1. **Read input**, build the garden matrix, compute the optimum, P and the chromosome length (sections 2 and 3).
2. **Initial population** (`generateChromosomes`): popSize chromosomes, each with 23 random genes from 0 to P − 1.
3. **Evaluate** every chromosome with `performRake` → fitness list + raked gardens. If generation 0 already contains the optimum, this population size is finished.
4. **GA loop** (`geneticAlgorithm`), every generation:
   - 4.1. Record max, mean and min fitness (for the graphs).
   - 4.2. Sort indices by fitness. If the best equals the optimum, or 1000 generations have passed, stop and return the best chromosome, the fitness list, the gardens and the history.
   - 4.3. **Elitism:** copy the 2 best chromosomes.
   - 4.4. **Selection:** pick popSize parents with the chosen method (tournament or rank).
   - 4.5. **Crossover:** pair neighbouring parents; two-point crossover with probability 0.7.
   - 4.6. **Mutation:** each gene of each child mutates with probability 0.1 (random or multiplicative).
   - 4.7. **New population** = 2 elites + the first popSize − 2 children.
   - 4.8. **Evaluate** the new population and continue with the next generation.
5. **Result:** the garden with the highest fitness over all population sizes is printed; `plotFitness` and `plotComparison` save the graphs as PNG files.

```
geneticAlgorithm(population, fitness):
    generation = 0
    loop:
        record max / mean / min (fitness)
        order = indices sorted by fitness, best first
        if fitness[order[0]] == maxFitness or generation == 1000:
            return best, fitness, gardens, history
        elites   = copies of population[order[0]], population[order[1]]
        parents  = selectTournament(k = 2)  or  selectRank
        children = crossover(parents)           # two-point, p = 0.7
        mutationFn(children)                     # p = 0.1 per gene
        population = elites + children[: popSize - 2]
        fitness, gardens = performRake(population)
        generation += 1
```

7. Creating a new generation

7.1 Elitism

The 2 best chromosomes of the current generation are copied unchanged into the next one. They skip crossover and mutation, so the best fitness never decreases: the max curves in all graphs are monotone.

7.2 Selection (two methods)

Both methods use only the **order** of fitness values, not their size, because the values lie close together. Each method returns popSize copies (`chromosome[:]`), so later mutation never changes a shared list.

Tournament selection (`selectTournament`, k = 2): for each slot, draw 2 different individuals at random and keep the fitter one. The best individual wins every tournament it enters; the worst never wins. With k = 2 the chance of being picked grows linearly with rank, like linear rank selection, while keeping enough diversity. As the assignment notes, tournaments of more than 3 leave the weaker half almost no chance.

Rank selection (`selectRank`): sort individuals from worst to best and give them ranks 1 … N. An individual is picked with probability

$$p(i) = rank(i) / (1 + 2 + \ldots + N) = 2 \cdot rank(i) / (N\,(N + 1))$$

For N = 50, the best individual has p = 3.9 % and the worst 0.08 %, regardless of how close their fitness values are.

7.3 Crossover

Two-point crossover (`crossover`) is applied to consecutive pairs of selected parents with probability 0.7; otherwise both parents pass unchanged. Two different cut points cut1 < cut2 are chosen from 1 … length − 1:

```
parent A: [a a a | a a a a | a a a]
parent B: [b b b | b b b b | b b b]
child 1:  [a a a | b b b b | a a a]
child 2:  [b b b | a a a a | b b b]
```

Two-point crossover was chosen because the order of genes matters: the first passes decide which cells later genes find free. With 1–2 cuts, consecutive blocks of good genes survive; many cuts (close to uniform crossover) would break them. Children need no repair, because duplicates and blocked entries are allowed. With an odd population, the last parent passes unchanged.

7.4 Mutation (two methods)

Each gene of each child mutates with probability 0.1, so about 2 of the 23 genes change per chromosome.

- **Random:** the gene gets a new uniform value from 0 to P − 1, i.e. a completely new entry point (and a random turn bit).
- **Multiplicative:** the gene is multiplied by a random factor from 0.7 to 1.3 and rounded; if the value did not change it is increased by 1, and it is capped at P − 1. A gene equal to 0 becomes a random value 1–3. Because neighbouring values are neighbouring entry points, this is a **local** move along the perimeter. Its step grows with the gene value: around gene 5 it moves by about ±1, around gene 40 by up to ±12.

8. Parameters

Two parameters are chosen at run time; the rest are constants in `game.py`.

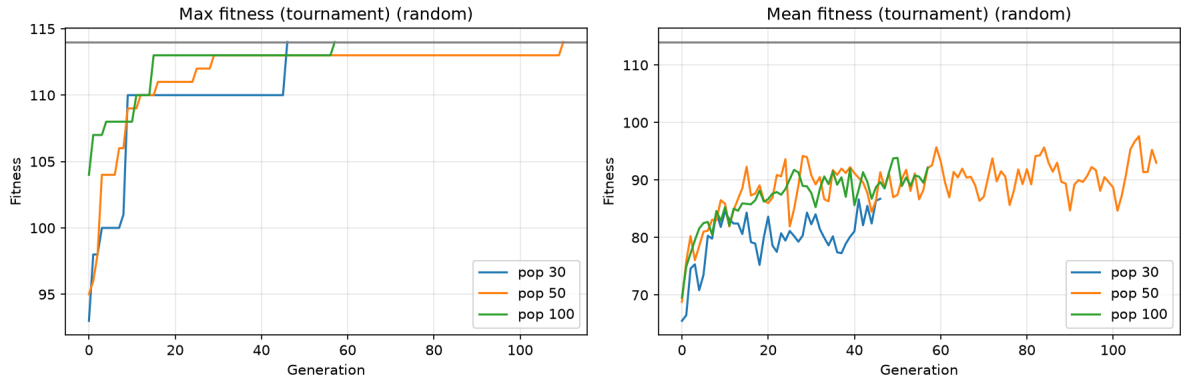
Parameter	Value	Where set
Selection method	tournament (0) or rank (1)	input
Mutation method	random (0) or multiplicative (1)	input
Population sizes	30, 50, 100 (one GA run each)	`POPULATIONS`
Chromosome length	max(3, int(0.85 · (W + H + stones))) = 23	`startGame`
Max generations	1000	`geneticAlgorithm`
Elites per generation	2	`geneticAlgorithm`
Tournament size k	2	`geneticAlgorithm`
Crossover	two-point, probability 0.7	`crossover`
Mutation probability	0.1 per gene	`mutationFn`
Multiplicative factor	0.7 – 1.3	`mutationFn`
Stop condition	fitness = W · H − stones, or 1000 generations	`geneticAlgorithm`

9. Experiments and comparison of settings

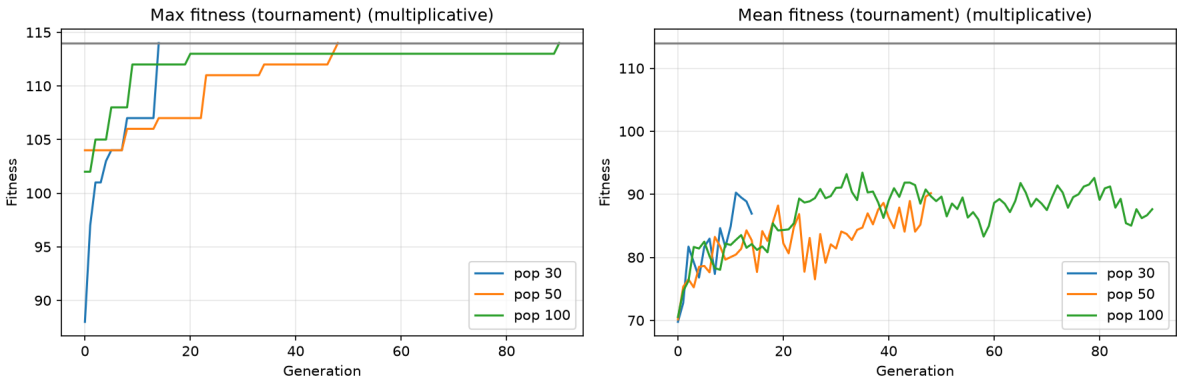
Population size matters most; random mutation is equal or better than multiplicative; the two selection methods perform almost the same. All experiments use the configuration from the assignment (section 2, optimum 114) and the final code (23 genes).

9.1 One run per method (graphs from the project folder)

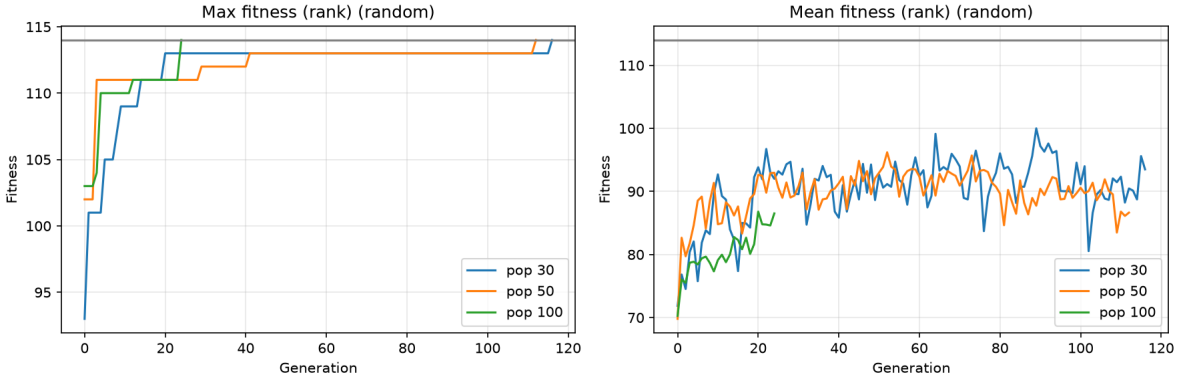
The program was run once for each of the four combinations of selection and mutation. Each run executes the GA for population sizes 30, 50 and 100 and saves the max and mean fitness per generation. All twelve GA runs reached the optimum of 114.



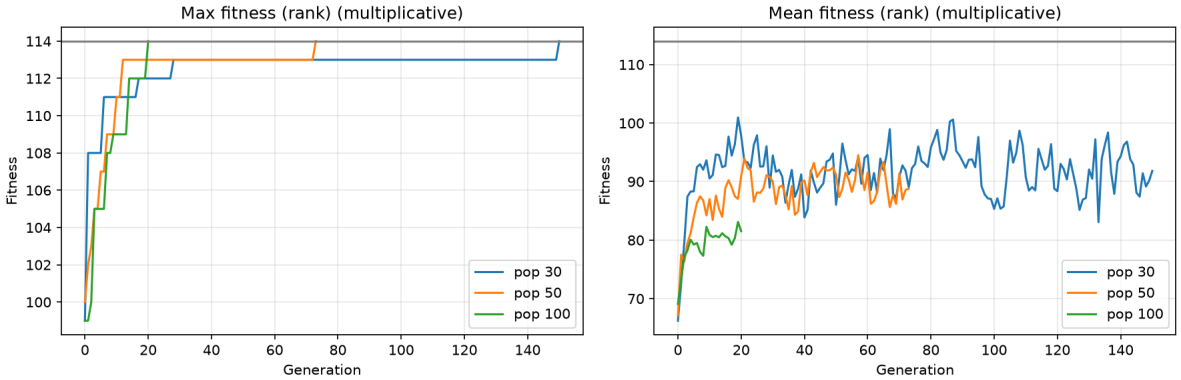
tournament selection + random mutation



tournament selection + multiplicative mutation



rank selection + random mutation



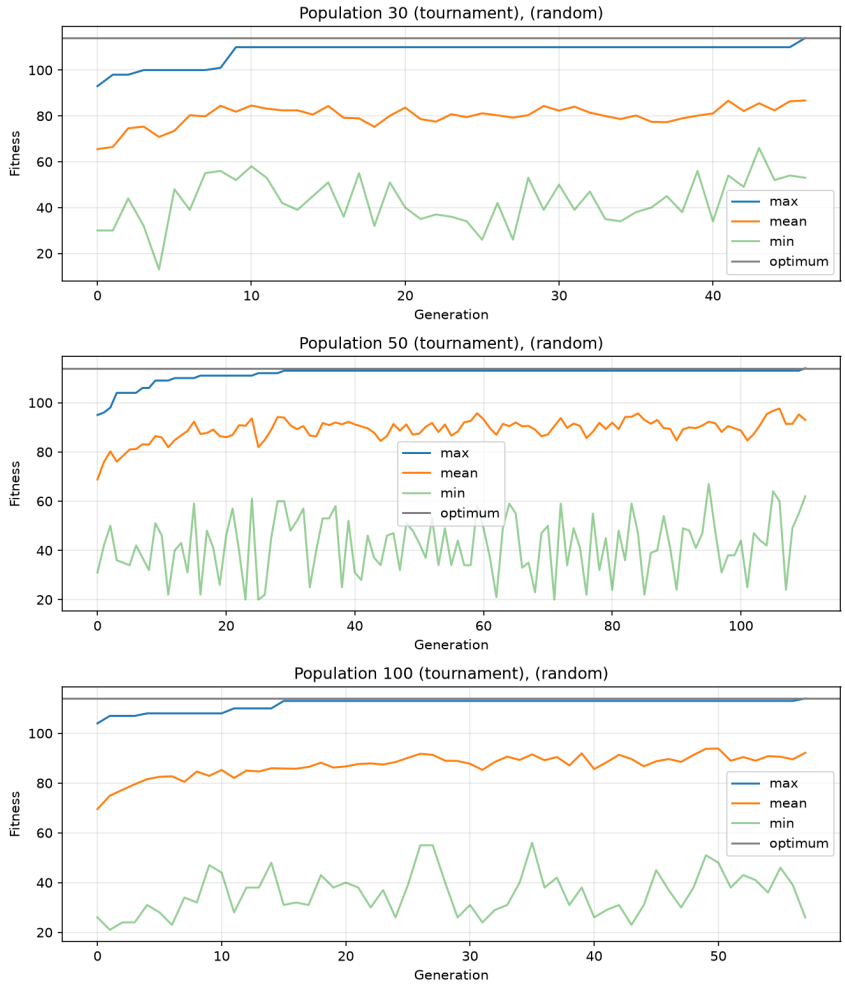
rank selection + multiplicative mutation

Generations needed to reach 114, read from the graphs:

Selection + mutation	**Pop 30**	**Pop 50**	**Pop 100**
tournament + random	≈ 46	≈ 110	≈ 57
tournament + multiplicative	≈ 14	≈ 48	≈ 90
rank + random	≈ 116	≈ 112	≈ 24
rank + multiplicative	≈ 150	≈ 73	≈ 20

- The max fitness rises in steps: elitism keeps the best individual until a better one appears. Most runs reach 110–113 within 20 generations and then spend most of the time on the last 1–3 cells.
- The mean fitness stays at 80–95, far below the max. The population stays diverse because of the mutation probability of 0.1 and the mild selection pressure of k = 2.
- Single runs vary strongly (14 to 150 generations), and here no method or population size is consistently the fastest. A fair comparison therefore needs many runs per setting (section 9.2).

Detailed view of one run (tournament + random), including the minimum fitness:



max, mean and min fitness per population size (tournament + random)

The minimum fitness stays at 20–70 in every generation: some children always get stuck early, which is the price of the strong mutation and of a fitness function that ends the game when the monk is stuck.

10. Evaluation of the system

The system reliably solves the assignment garden; its weak point is the last 1–3 cells, where runs with small populations can stall.

Strengths

- **Correct and deterministic simulation.** One chromosome always gives the same raking and fitness, and every pass ends after at most W · H steps.

- **Reliable on the required garden:** with population 100 and random mutation, runs reached 114/114, and all twelve runs saved in the project folder succeeded.
- **Compact encoding.** Entry points and turn decisions share the same 23 genes, below the limit of 28. Any chromosome is valid, so crossover and mutation need no repair.
- **Any map.** Garden size and stones are entered by the user; nothing in the code is specific to the example.

Limitations

- **Coupled genes.** Changing one gene changes both an entry point and a turn bit used elsewhere, so a small mutation can have a large effect.
- **Multiplicative mutation is scale-dependent:** genes near 0 move by about 1, genes near P by up to 30 %, so the left edge is explored much more finely than the top edge.
- **Unreachable cells are not detected.** On a map where a free cell cannot be reached by any straight pass (e.g. boxed in by stones), the optimum W · H − stones is impossible and the GA runs all 1000 generations.
- **Input:** non-numeric width, height, stone count or method number crashes the program, and a stone entered twice is counted twice, so the optimum can no longer be reached.

11. Improvements and tuning during development

The solution went through several versions; each fix below came from a failure observed while testing.

#	Problem observed	Cause	Fix
1	Garden already raked when the next chromosome was evaluated	one shared matrix for the whole population	a fresh copy of the garden per chromosome
2	Infinite loop inside one pass, wrong fitness	cells were marked with the gene value, so gene 0 wrote 0 = unraked	cells are marked with a pass counter starting at 1
3	Turn choice not reproducible	the choice did not depend on the chromosome	decision = `chromosome[t] mod 2`
4	Tournament picked the wrong individual; mutation changed several individuals at once	`fitness.index(value)` returned the first equal value; lists were shared	selection by index; every selected chromosome is copied
5	Best solution lost; fitness belonged to other individuals	no elitism; the crossed list was evaluated instead of the new population	2 elites; the new population is built first, then evaluated
6	Results not comparable between runs	random chromosome length `randint(4, max)`	fixed length 0.75 · max, later raised to 0.85 · max (23 genes)
7	Rarely reached the optimum	population 5, 20–100 generations	populations 30/50/100, limit 1000 generations
8	Crash when entering a single number as a stone	the second value was read before the length check	the length is checked first
9	Graphs not comparable	saved runs came from different test gardens	all graphs regenerated on the assignment configuration

Tuning. After the program was correct, a second mutation operator (multiplicative) was added, and both were compared with both selection methods and three population sizes (section 9). The first

finding was applied: the chromosome length factor was raised from 0.75 to 0.85, which raised the overall success. The experiments suggest further tuning that the current constants do not yet use:

- use the full allowed chromosome length (28 genes for 12 × 10 garden).
- prefer random mutation and population 100 for the most reliable result.

12. Running the program and example output

Requirements: Python 3.12+ and matplotlib (`pyproject.toml`; install with `uv sync` or `pip install matplotlib`). Run with `python main.py` and enter the configuration from section 2.

Output:

- the matrix of the best garden over all population sizes; −1 = stone, k = cell raked in pass k,
- `fitness-<selection>-<mutation>.png`: max, mean and min fitness per generation, one panel per population size,
- `comparison-<selection>-<mutation>.png`: max and mean fitness of all population sizes in one graph.

Full solution of the assignment garden (tournament + random mutation, 114/114 cells):

```
12 8 9 9 9 9 7 4 10 10 3 11

12 8 8 8 8 -1 7 4 10 10 3 11

12 -1 8 8 8 8 7 4 10 10 3 5

8 8 8 8 -1 8 7 4 10 10 3 5

13 13 -1 8 8 8 7 4 10 10 3 5

7 7 7 7 7 7 7 4 10 10 3 5

4 4 4 4 4 4 4 4 -1 -1 3 3

1 1 1 1 1 1 1 1 1 1 1 1

2 2 2 2 2 2 2 2 2 2 2 2

6 6 6 6 6 6 6 6 6 6 6 6
```

Reading it:

- Pass 1, 2 and 6 rake the whole of row 8, 9, 10.
- Pass 3 enters row 7 from the right, is blocked by stone, chooses up direction (can't go down because of 1) and goes up till the end of garden.
- Pass 4 enters row 7 from the left and faces the stone at (8, 6). Bottom row is already raked, so it is forced up column 7 and leaves at the top.

This logic applies for all other passes, while, finally, pass number 13 fills the last uncovered cell.